Functional programming

CS 22B, Spring 2026

Module 03: Function

Presented by Jessica Huynh-Westfall, Ph.D.

02

AGENDA

- Review functions
- Functional Programming
  - Functional Programming
  - Side effect
  - Pure function
  - Higher-order functions
  - Lambda function

03

Chapter 9: Functions

The Quick Python Book

Syntax for defining a function

define — name of the function — parameter(s) pass into function

```
def get_at_content (dna):
    '''Function computes the proportion of A's and T's in dna.
     Args:
          dna: string param
    Returns:
          at_content: type(float) between 0 and 1, inclusively
    '''
    # length of arg
    length = len(dna)
    a_count = dna.count('A')
    t_count = dna.count('T')
    at_content = (a_count + t_count) / length
    return at_content
```

docstring

Obtain value by printing `get_at_content.__doc__`

return will create the outputs of the function

Function parameter options

05

- You can specify the specific type of an argument

```
def get_at_content(dna:str):
    '''Function computes the proportion of A's and T's in dna.
    Args:
        dna: string param
    Returns:
        at_content: type(float) between 0 and 1, inclusively
    '''
    length = len(dna)
    a_count = dna.count('A')
    t_count = dna.count('T')
    at_content = (a_count + t_count) / length
    return at_content
```

Function parameter options

- You can specify the specific type of an argument
- You can specify if you want the returned object to be of a specific type

```
def get_at_content(dna:str)-> int:
    '''Function computes the proportion of A's and T's in dna.
     Args:
          dna: string param
    Returns:
          at_content: type(float) between 0 and 1, inclusively
    '''
    length = len(dna)
    a_count = dna.count('A')
    t_count = dna.count('T')
    at_content = (a_count + t_count) / length
    return at_content
```

Question

```
print("AT content is " + get_at_content("ATGACTGGACCA"))
```

If get_at_content already defined what will be the output to the above statement?

- □ A) AT content is 0.5
- □ B) AT content is
- □ C) Nothing will be printed
- ■ D) Error

Assign values to arguments in functions

08

```
def get_at_content(dna, sig_fig=2):
    '''Function computes the proportion of A's and T's in dna.
    Args:
         dna: string param
    Returns:
        at_content: type(float) between 0 and 1, inclusively
    '''
    length = len(dna)
    a_count = dna.count('A')
    t_count = dna.count('T')
    at_content = (a_count + t_count) / length
    return round(at_content)
```

```
>>>dna = "ATGACTGGACCA"
>>>print("AT content is " + str(get_at_content $ig_fig=3, dna=dna,)))
```

keyword passing

Variable number of arguments; *

09

```
def varArg(x, *y):
    '''Example to show variable arguments'''
    print(x)
    print(y)
    print(type(y))

varArg(1)
varArg(1,2)
varArg(1, 2, 3, 4)
```

* causes all excess non-keyword arguments to be collected and assigned as **tuple**

Variable number of arguments; **

```
def varArg(x, **y):
    '''Example to show variable arguments'''
    print(x)
    print(y)
    print(type(y))
    print(y.keys())
    print(y.values())

varArg(x=1)
varArg(1)
varArg(1,z=2)
varArg(1, a=2,b= 3, c=4)
```

** causes all excess keyword arguments to be collected and assigned as **dictionary**

What about…

```
varArg(1 , 2, a=3, b=4)
```

Mixed argument-passing techniques

It is possible to use all the argument-passing features of Python functions at the same time with the following rules:

1. Positional arguments come first
2. then named arguments
3. followed by indefinite positional arguments with a single *
4. and last of all the indefinite keyword argument with **

11

Functional programming

Overview of Functional Programming

What is Functional Python Programming?

Functional programming is a style where:

- Functions do NOT change outside variables
- Functions always return the same output for the same input
- Avoid modifying data (no hidden state changes)

Procedural vs functional programming

Procedural

The variable changes inside the for loop

```
x = 0
for i in range(10):
    x = x + i
print(x)
```

x keeps changing (mutable state)

Functional

Does not use state.

```
x = sum(range(10))
print(x)
```

x does not change (not mutable)
Use built-in function (eg., sum())
Has no side effect

Functional programming minimizes or isolate side effects

Function with side effect

The original variable changes

```
def add_items(lst):
    lst.append("A")
    return lst

my_list = [1, 2, 3]
add_items(my_list)
print(my_list)
```

Original list is changed

```
[1, 2, 3, 'A']
```

Pure function

The original variable remains unchanged

```
def add_items(lst):
    return lst + ["A"]

my_list = [1, 2, 3]
new_list = add_items(my_list)

print(my_list)
print(new_list)
```

Class exercise

CL 7.1

Are functions f1 and f2 pure?

```
def f1(x):
    print(x)
    return x * 2

def f2(x):
    return x * 2
```

Technically, printing changes a program output and thus is a side effect.

A function is NOT pure if:

- Prints, reads, or writes a file
- Modifies a global variable
- Changes a variable passed into it
- Dependent on external variables

Function should be Reproducible

A function should always return the same value if given the same input. The code below breaks the rule because the behavior of the function changes depending on the value of to_add at the time that it's called.

```
def my_function(x):
    return(x + to_add)
```

```
>>>to_add =['a', 'b', 'c']
>>>x = [1,2,3]
>>>print(my_function(x))
[1, 2, 3, 'a', 'b', 'c']
>>>to_add =['x', 'y', 'z']
>>>print(my_function(x))
[1, 2, 3, 'x', 'y', 'z']
```

Class exercise

CL 7.2

How do these two functions differ in terms of mutability?

```
lsti = [1, 2, 3]
def my_function1(i):
    i.extend(['a', 'b', 'c'])
    return(i)
lst = my_function1(lsti)
```

Function1 modifies the input, `lsti`

```
def my_function2(i):
    return(i + ['a', 'b', 'c'])
```

Function2 is immutable and *functional*

Class exercise

CL 7.3

Change this to make it functional

```
total = 0
for i in range(1, 11):
    total += i*i
print(total)
```

2/17: Start here next class

Mutable Objects as Arguments

Passing a mutable objects as an argument in a function allows the function to modify the original object in place and these changes are reflected outside the function.

```
def f(n, list1, list2):
    list1.append(3)
    list2 = [4, 5, 6]
    n = n + 1


x = 5
y = [1, 2]
z = [4, 5]
f(x, y, z)
```

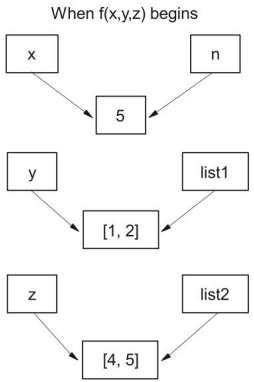

Figure 9.1 At the beginning of function `f()`, both the initial variables and the function parameters refer to the same objects.

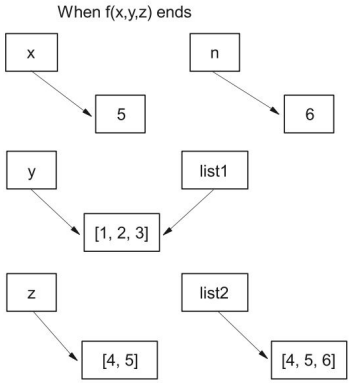

Figure 9.2 At the end of function `f()`, `y` (`list1` inside the function) has been changed internally, whereas `n` and `list2` refer to different objects.

Modification of y is a “side effect”

Side effect of function

If the function has side effects, then we have no way of knowing what/why the value of x is after the function call without going and looking at the code. Functional programming is used so that data is immutable and pure functions ensure predictability and reusability!

```
x = [1,4,9]
some_function(x)
```

What is the value of x now?

```
def some_function(input):
    some_other_function(input)
    another_function(input)
    yet_another_function(input)
```

What if in some_function() we find it calls other functions?

Recursion method to make code function

Recursive has no mutable variables and uses function calls instead of loops.

```
from functools import reduce
result = reduce(lambda acc, x: acc + x, range(10))
```

```
def sum_numbers(x):
    if x == 0:
        return 0
    return x + sum_numbers(x - 1)
print(sum_numbers(9))
```

Base case:
Condition stops recursion when base case met

Recursive case:
Call function with (x-1)

Higher Order Functions

Higher order function

There are many common programming tasks that can be made easier using higher order functions. A higher-order function is a function that either:

- Takes another function as an argument (input), OR
- Returns a function as a result

This allows for function composition, reusability, and cleaner code. Examples of built-in order functions include:

- map(function, iterable) and apply()
- filter(function, iterable)
- sorted(iterable, key=function)

What is map() and apply()?

Python functions used to transform or manipulate data, especially when working with pandas (we will introduce this later in the semester).

- They are commonly used to apply a function to elements within a sequence (like lists or DataFrame columns)
- **`map()`** used to apply a function to each element of a sequence (list, pandas Series)
- `apply()` used on both Series and DataFrames. Apply functions element-wise, row-wise, or column-wise.

  (We will come back to `apply()`)

More on map(function, iterable)

map(function, iterable) returns a map object (an iterable like a list) of the results after applying the given function to each item of a given iterable (list, pandas Series, etc.).

map() takes two arguments:

1. function: The name of the transformation function to apply
2. iterable: The sequence (e.g., list, Series) you want to iterate over.

Class exercise


```
def square(x):
    return x ** 2
numbers = [1, 2, 3, 4]
squared_numbers = list(map(square, numbers))
print(squared_numbers)
```

CL 7.4

When we use map() on a list:

- *What is the expected output of squared_numbers?*
- *What will numbers returned after mapping?*

map() with lambda function

The transformation functions that we pass to map() are often very short, it's quite common to use lambda functions to do the job instead.

lambda function: A lambda function can take any number of arguments, but can only have one expression

```
lambda arguments: expression
```

- arguments: The parameters you pass to the lambda function
- expression: A single expression whose result is automatically returned

Class exercise


```
stringlist = ["and", "ant", "air", "anything"]

bstring = list(map(lambda x:x.replace("a", "B"),stringlist))
print(bstring)
```

CL 7.5

When we use map() and lambda function:

- *What is the expected output?*

apply() on Series and DataFrame

apply() used on both Series and DataFrames. Apply functions element-wise, row-wise, or column-wise.

Syntax for Series:

Series.apply(function)

Syntax for DataFrame: (we will revisit this later)

DataFrame.apply(function, axis=0 or 1), where:

- axis=0: Apply function to each column (default)
- axis=1: Apply function to each row

filter(function, iterable) is for selecting elements from a list that satisfy some condition and returns a *filter object* (an iterable)

filter() takes two arguments:

1. The **name of a function** that takes a single element as its argument and returns a Boolean value (True or False) to indicate whether or not that element should be included in the result list
2. The **name of the original list**

filter() function

Using filter() function to filter elements based on a condition. With lambda, apply a condition

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers)
```

Class exercise

Given a list of numbers, remove any numbers whose square are less than 100.

```
num_input= [1, 4, 5, 20, 15, 70, 15, 6, 7, 70]

def greater100(num):
    return num ** 2 >= 100
```

CL 7.6

Work with a partner and write your own code with at least 1 higher order function.

sorted() is for sorting lists in alphabetical or numerical order and it returns a new list

- It creates a sorted copy of the list, which is more compatible with the functional programming ideas of avoiding state and mutability.
  - Returns a new list while the original list is unchanged
- sorted() method is more flexible as it's not restricted to lists – we can call sorted on any iterable data type (strings, files, etc).
- sorted() can also sort in the reverse order.

sorted() to sort elements

sorted() function

Using sorted() function to sort a list of tuples by the second element

```
my_list = [4, 1, 9, 2, 4]
sorted_list = sorted(my_list)
print(sorted_list)
```

```
numbers = [5, 2, 9, 1, 7]
sorted_numbers_desc = sorted(numbers, reverse=True)
print(sorted_numbers_desc)
```

sorted() function with keys

sorted() function can be used to sort in a different order – for example, by length.

To do this, we supply sorted() with a **key function**. The key function must take a single argument, and return the value that we want to sort on.

```
data = [("james", 30), ("mai", 20), ("ann", 19)]
sorted_data = sorted(data, key=lambda x: x[1])
```

Class exercise


```
vegfruit = [(1, "zuchinni"), (3, "carrot"), (2, "apple"),
(5, "orange"), (4, "banana")]
```

CL 7.7

Sort this list of tuples by second element in reverse order

Class exercise

CL 7.6

Rewrite this procedurally written code into functional style using map(), lambda, sum()

```
total = 0
for i in range(1, 21):
    if i % 2 == 0:
        total += i**2
    else:
        total += i**3

print(total)
```